

# React Framework Speaker Notes

University Lecture • Target Duration: 2 Hours (120 Mins)

Module: Frontend Architecture  
21 Slides Total

## Slide 1: Title & Introduction

 5 mins (0:00 - 0:05)

### FOCUS

Set the stage and manage expectations.

**WELCOME** "Welcome everyone. Over the past few weeks, you've mastered HTML, CSS, and Vanilla JavaScript. You know how to build a web page and make it interactive."

"Today, we are making a massive paradigm shift. We are looking at React, the framework created by Facebook, which is the industry standard for building modern web applications."

"The goal today isn't to memorize syntax—you can look that up later. The goal is to understand **how React thinks** and why it fundamentally changes how we build UIs."

## Slide 2: The Baseline (Vanilla JS Problem)

 7 mins (0:05 - 0:12)

### KEY CONCEPT

Imperative programming vs the DOM.

**REFERENCE** "Let's look at the code on the screen. This should look very familiar to you from your recent assignments."

"To update a simple counter in vanilla JS, we have to: query the button, query the text label, define a variable, add a listener, increment the variable, manually format a string, inject it into the DOM, and then check a condition to manually add a CSS class."

**The problem:** "This is *imperative* programming. We are micromanaging the DOM step-by-step. In a large app, state (our data) and the UI get out of sync easily. It turns into spaghetti code."

## Slide 3: The React Philosophy

 5 mins (0:12 - 0:17)

### ANALOGY

Taxi vs driving yourself (Declarative).

**Declarative** "Instead of telling the browser *how* to update the UI step-by-step, React lets us declare *what* the UI should look like based on current data. React figures out the steps."

- **Component-Based:** We break the page down into Lego bricks. A header, a sidebar, a post, a like button.
- **Reactive:** If the data changes, the UI updates automatically. You never manually touch the DOM again.

## Slide 4: Components are JS Functions

 6 mins (0:17 - 0:23)

### HIGHLIGHT

Capitalized function names.

"This is the most important secret of React: **Components are just standard JavaScript functions.**"

"Look at the code. `function ProfileCard()`. The only difference between this and the math functions you wrote last week is that this function returns UI elements."

**NOTE** "Always capitalize your component names. If you write `profileCard()`, React thinks it's a standard HTML tag and your app will break."

## Slide 5: HTML vs. JSX

 8 mins (0:23 - 0:31)

### SYNTAX TRAP

class vs className.

"Wait, HTML inside JavaScript? Yes. This is called JSX (JavaScript XML)."

"Under the hood, JSX is just syntactic sugar. Because it's actually JavaScript, there are a few rules that differ from standard HTML:"

- **className:** You cannot use `class="container"` because `class` is a reserved keyword in JS. You must use `className`.
- **Closing tags:** Every tag MUST close. `<img>` will crash your app. It must be `<img />`.
- **Inline styles:** They are passed as JS objects, hence the double curly braces `{{ }}` and camelCase properties like `fontSize`.

## Slide 6: Props as Function Arguments

 7 mins (0:31 - 0:38)

### KEY RULE

Props are read-only.

"If components are just functions, how do we pass data to them? Arguments. In React, these arguments are called **Props** (Properties)."

"In the parent `App`, it looks like we are setting HTML attributes: `role='Admin'`. But under the hood, React packages those up into a single object and passes it to the `UserBadge` function."

**CRITICAL** "Data only flows one way: Down. A child component can NEVER modify its own props. They are strictly read-only."

## Slide 7: Event Handling

 5 mins (0:38 - 0:43)

## Slide 8: Component State (useState)

 10 mins (0:43 - 0:53)

### DEEP DIVE

Explain the array destructuring syntax.

**STATE** "This is the heartbeat of React. If props are arguments passed from above, **State** is a component's personal, internal memory."

"We use the `useState` hook. It returns an array with two things: the current value (`count`), and a function to change that value (`setCount`)."

"Why can't we just use a normal `let count = 0` variable? Because React won't know it changed! When you call `setCount()`, you are ringing a bell that tells React: *'Hey, my data changed, please re-run my function and update the UI!'*"

## Slide 9: The Virtual DOM

 6 mins (0:53 - 0:59)

### PERFORMANCE

Why React is so fast.

"When you call a state setter, how does React update the page without lagging the browser? Touching the real DOM is notoriously slow."

"React keeps a lightweight copy of the UI in memory—the Virtual DOM. When state changes, it generates a new Virtual DOM, compares it to the old one (Diffing), and figures out the exact minimal changes needed."

"It patches the real browser DOM surgically. You just write the logic; React handles the optimization."

## Slide 10: Conditional Rendering

 5 mins (0:59 - 1:04)

### JS NATIVE LOGIC

Ternary and Logical AND.

"Because JSX is just JavaScript, we don't need special framework-specific directives like `v-if` or `ng-show`. We just use standard JS operators."

"Use the ternary operator `?:` when you want to show one thing OR another."

"Use the logical AND operator `&&` when you want to show something OR nothing at all."

## Slide 11: Lists & Keys

 6 mins (1:04 - 1:10)

### INTERVIEW Q

"Why do we need keys?"

"To render a list of data, we use the standard JavaScript `Array.map()` method to transform an array of data objects into an array of JSX"

## Slide 12: Forms & Controlled Components

 7 mins (1:10 - 1:17)

### CONCEPT

React taking over the DOM.

"In HTML, form inputs like `<input type='text' value='' />` naturally keep their own state. You type, the box updates."

"In React, we want **Single Source of Truth**. We bind the input's `value` directly to our React state, and update that state on every keystroke via `onChange`. React is now driving the input box, not the browser DOM."

"This is called a *Controlled Component*. It makes validating forms incredibly easy."

## Slide 13: Component Lifecycle

 5 mins (1:17 - 1:22)

### TIMELINE

Mount -> Update -> Unmount

"A component has a lifecycle, just like a program. It is created and inserted into the DOM (Mounting)."

"It lives there, reacting to state and prop changes (Updating)."

"Eventually, the user navigates away, and it is removed from the DOM (Unmounting). We need to understand this to know when to fetch data or clean up event listeners."

## Slide 14: Side Effects (useEffect)

 10 mins (1:22 - 1:32)

### WARNING

The dependency array trap.

**ADVANCED** "This is the most complex core hook. `useEffect` is for 'Side Effects'—things that reach outside of React. Fetching an API, setting up a WebSocket, modifying `document.title`."

"The second argument is the **Dependency Array**. It is critical:"

- No array: Runs after *every single render* (Careful, infinite loops!).
- Empty array `[]`: Runs exactly once when the component mounts. (Perfect for initial API fetches).
- Variables inside `[userId]`: Runs on mount, AND whenever `userId` changes.

## Slide 15: The Context API

 6 mins (1:32 - 1:38)

### PROBLEM SOLVED

Prop Drilling.

"Imagine passing a `theme` or `userProfile` prop down through 10 layers of components just so the very bottom one can use it. That's called Prop Drilling. It's messy."

## Slide 16: Custom Hooks

 6 mins (1:38 - 1:44)

### CODE REUSE

Extracting stateful logic.

"If you have complex logic—like tracking the window width or fetching data—you don't want to copy-paste that ``useState`` and ``useEffect`` logic into 5 different components."

"You can create your own Custom Hook. It's just a normal JavaScript function that starts with the word ``use``. It packages up stateful logic so you can share it cleanly."

## Slide 17: Thinking in React

 7 mins (1:44 - 1:51)

### WORKFLOW

The architectural mindset.

"How do you actually start building an app? You don't just write code. You follow this process:"

- **1. Break it down:** Look at the design mockup and draw boxes around everything. Each box is a component.
- **2. Build Static:** Build the UI with hardcoded data. No state, no interactivity yet.
- **3. Find State:** Figure out the absolute minimum state you need (e.g., a search query string).
- **4. Data Flow:** Wire the state to the inputs so the UI updates.

## Slide 18: Ecosystem (Redux/Zustand)

 4 mins (1:51 - 1:55)

### LOOKING AHEAD

Beyond local state.

"Context is great, but for massive enterprise applications, you need dedicated State Management libraries."

"**Redux** is the old heavyweight champion—very strict. **Zustand** is the modern, lightweight favorite. **React Query** is mandatory for anyone dealing with complex API fetching. You'll encounter these in your future jobs."

## Slide 19: Meta-Frameworks

 4 mins (1:55 - 1:59)

### INDUSTRY STANDARD

Next.js and Vite.

"React is technically a 'library', not a full framework. It only handles the UI. It doesn't handle routing or server-side rendering."

"In the real world, you rarely use pure React. You use a Meta-framework like **Next.js** for production apps, or tools like **Vite** for lightning fast local development (which we will use in our job)."

## Slide 20: Recap

 2 mins (1:59 - 2:01)

### SUMMARY

The 4 core pillars.

"To summarize 2 hours of theory into four words:"

- **UI:** Built with pure JavaScript functions returning JSX.
- **Props:** How data flows down the tree.
- **State:** The local memory that triggers re-renders.
- **Effects:** How we sync with the outside world.

## Slide 21: Lab Time

 3 mins (2:01 - 2:04)

### ACTION ITEM

Transition to coding.

"Theory only gets you so far. It's time to build."

"Please open your terminals. We are going to use Vite to scaffold our first React application. Run the command on the board: ``npm create vite@latest my-app -- --template react``."

**INSTRUCT** "Once you have it running, your first task is to delete the boilerplate App component and build a simple Counter button using ``useState``. I will be walking around to help."

## Instructor Notes: Anticipated Student Questions

### Q: Why does my component re-render infinitely?

A: Usually, they mutated state directly inside the render body instead of inside an event handler or ``useEffect``. Or, they forgot the dependency array on a ``useEffect`` that updates state.

### Q: Why is my state update one step behind in ``console.log``?

A: Explain that ``setState`` is asynchronous/batched. Logging state immediately after setting it will log the old closure value. (Classic beginner trap!).

### Q: Do I still need to learn HTML/CSS?

A: Absolutely. JSX compiles to HTML. If you write bad semantic HTML in JSX, it's still bad HTML. React just changes how it gets onto the screen.